

Member ID: _____

Time: _____

Rank: _____



Java Programming

(340)

STATE 2025

PRODUCTION:

AIChatBotGenerator_State

_____ (680 points)

Test Time: 90 minutes

GENERAL GUIDELINES:

Failure to adhere to any of the following rules will result in disqualification:

1. Member must hand in this test booklet and all printouts if any. Failure to do so will result in disqualification.
2. No equipment, supplies, or materials other than those specified for this event are allowed in the testing area. No previous BPA tests and/or sample tests (handwritten, photocopied, or keyed) are allowed in the testing area.
3. Electronic devices will be monitored according to ACT standards.

You will have ninety (90) minutes to complete your work.

Your name and/or school name should *not* appear on work you submit for grading.

1. Create a folder on the flash drive provided using your contestant number as the name of the folder.
2. Copy your entire solution/project into this folder. The project folder for you has already been provided: Java_State
3. Submit your entire solution/project so that the graders may open your project to review the source code.
4. Ensure that the files required to run your program are present and will execute on the flash drive provided.
5. You will need to use a local Java IDE to complete this exam.

*Note that the flash drive letter may *not* be the same when the program is graded as it was when you created the program.

*It is recommended that you use relative paths rather than absolute paths to ensure that the program will run regardless of the flash drive letter. It is HIGHLY recommended that you place all of the files into one folder.

The graders will *not* compile or alter your source code to correct for this.
Submissions that do *not* contain source code will *not* be graded.

Assumptions to make when taking this assessment:

- One Java file is provided in the contest folder (AIChatBotGenerator_State).
- The goal of the program is to give the user a menu system where they can create a list of random chatbots, manually generate a chatbot, give the user the ability to search for a language from the list of chatbots, and give the user the ability to exit the program.
- Each chatbot record will be organized by primary language, secondary language, token rate, and algorithm complexity.
- The random chatbot generation and printing methods have already been completed for you.
- The user will press 1 to create random chatbots (1 to 50), press 2 to create a single chatbot, press 3 to search for chatbots that have a certain language, and press 4 to quit.
- The Token Rate must be in the range of 5 to 20 (inclusive).
- The program must detect data value entry errors and range violations.
- When the user chooses option 4, the program must exit.
- All randomly and manually created chatbots will be added to the same list.
- Manually created chatbots need their languages adjusted so the first letter is capital and all following letters are lowercase, i.e.: Spanish, Japanese, etc.
- The Complexity will be a random double value greater than or equal to 2.0 and less than 9.1.
- The program will find the record with the lowest and a record for the highest Complexity.

Development Standards:

- Your Code must use a consistent variable naming convention.
- All subroutines (if any), functions (if any), and methods (if any) must be documented with comments explaining the purpose of the method, the input parameters (if any), and the output (if any). Readability is a goal of good code.

Commenting for Source Code Review (see the rubric):

- Certain sections of your code will be graded. These gradable blocks of code can range from creating data structures, method algorithms, exception handling, and class construction.
- The grading rubric contains a section called Source Code Review: in this section are listed a description all of the graded programming concepts.
- Each gradable item must have a comment listed at its beginning, and the comment must be prefixed with the comment flag. The flag helps the graders easily locate the code to increase the effectiveness of grading.
- The flag will always use this naming convention: **SC#** (NOTE: the # symbol will be replaced with sequential numbering, i.e. **SC1, SC2, SC3**, etc.
- No explanation in the comment with the flag is required, only the comment flag; however, any information placed in the comment could help the grader better understand and avoid any costly errors.
- The comment flag needs to be placed in close proximity to the block of code it represents.
- If a comment flag is not present, you will not receive credit.
- In this example, the Source Code Review has a gradable section of code for printing to the console (remember these are non-related examples):
 - SC12: *print* method in the *main* class is printing the correct object ____ 10 pts
 - The user will place the code above the method call:

```
//SC12 printing the car object  
System.out.println(car);
```

Java_AIChatBotGenerator_State

The goal of the program is to give the user a menu system where they can create a list of random chatbots with “Primary Language” and “Secondary Language”, “Token Rate”, and a “Complexity” rating. The program will allow the user to be able to have control options through key board entries. There are three major actions of the program: (1) Randomly create chatbot list; (2) Manually create chatbots; (3) search for a specific language; (4) close the program..

Input/Output Option [1]

All user input and output will take place in the console.

STEP 1 Option [1]: The program prompts the user to make a selection. This example selects #1.

```
Welcome to the BPA Chatbot Generator. Choose one of the options:
Press [1] to create random chatbots OR Press [2] to create a single chatbot
OR Press [3] to search by language OR Press [4] to end the program
```

STEP 2 Option [1]: After entering 1, the program will ask the user to enter in a value between 1 and 50 (this number is how many chatbots the program will create). After entering the amount of chatbot records to create, the program will list out the results. In addition, the program also finds the records with the highest Complexity and the least Complexity.

```
Welcome to the BPA Chatbot Generator. Choose one of the options:
Press [1] to create random chatbots OR Press [2] to create a single chatbot
OR Press [3] to search by language OR Press [4] to end the program
1
Please enter in a value between 1 and 50: 5
```

No.	Supported Languages	Token Rate	Complexity
1	Swedish, Hindi	20	8.3010
2	Russian, Hungarian	11	7.3139
3	Spanish, Greek	10	7.3877
4	Vietnamese, Greek	19	5.7438
5	English, Japanese	16	7.4068

```
Most Expensive Complexity Rate: 8.301 Swedish, Hindi

Least Expensive Complexity Rate: 5.7438 Vietnamese, Greek
```

After the list is created, the program returns to STEP 1. See the image above for this step.

STEP 1 Option [2]: The program prompts the user to make a selection. This example selects #2.

STEP 2 Option [2]: The user enters in the primary and secondary languages (two separate entries) (image to the right).

```
Welcome to the BPA Chatbot Generator. Choose one of the options:
Press [1] to create random chatbots OR Press [2] to create a single chatbot
OR Press [3] to search by language OR Press [4] to end the program
2
You are now creating a single chatbot
Enter in the native language
Nubian
Enter in the secondary language
Cuman
```

STEP 3 Option [2]: The user will enter in the token rate and complexity (two separate entries) (see image below).

```
Enter the token rate value (5 to 20):
11
Enter in the token complexity in its proper format (#####) in a range of 2.0 to 9.0
3.3
```

STEP 4 Option [2]: The output with the custom bot will be auto generated (see image below); the custom bot will appear in the list at the end if Option [1] is run first, else it will be at the top if Option [2] is run first. The program returns back to STEP 1.

No.	Supported Languages	Token Rate	Complexity
1	Swedish, Hindi	20	8.3010
2	Russian, Hungarian	11	7.3139
3	Spanish, Greek	10	7.3877
4	Vietnamese, Greek	19	5.7438
5	English, Japanese	16	7.4068
6	Cuman, Nubian	11	3.3000

STEP 1 Option [3]: The program prompts the user to make a selection. This example selects 3.

STEP 2 Option [3]: The user will enter in which language they are looking for. The user can technically enter in any letter of the language they are looking for, and have a language return the results that have a matching string. The search is NOT case sensitive. If there is no language with a string match then the program will inform the user “No chatbot found for the specified language.” The image on the right shows the different options.

STEP 1 Option [4]:

The user selects to close the program, it will give the message “Goodbye!” before stopping. See image below.

```
Welcome to the BPA Chatbot Generator. Choose one of the options:
Press [1] to create random chatbots OR Press [2] to create a single chatbot
OR Press [3] to search by language OR Press [4] to end the program
3
Enter the language you want to search for:
greek
Chatbot found: Spanish, Greek | Token Rate: 10 | Complexity: 7.3877
Chatbot found: Vietnamese, Greek | Token Rate: 19 | Complexity: 5.7438

Welcome to the BPA Chatbot Generator. Choose one of the options:
Press [1] to create random chatbots OR Press [2] to create a single chatbot
OR Press [3] to search by language OR Press [4] to end the program
3
Enter the language you want to search for:
gree
Chatbot found: Spanish, Greek | Token Rate: 10 | Complexity: 7.3877
Chatbot found: Vietnamese, Greek | Token Rate: 19 | Complexity: 5.7438

Welcome to the BPA Chatbot Generator. Choose one of the options:
Press [1] to create random chatbots OR Press [2] to create a single chatbot
OR Press [3] to search by language OR Press [4] to end the program
3
Enter the language you want to search for:
8
Chatbot found: Russian, Hungarian | Token Rate: 11 | Complexity: 7.3139
Chatbot found: Spanish, Greek | Token Rate: 10 | Complexity: 7.3877
Chatbot found: Vietnamese, Greek | Token Rate: 19 | Complexity: 5.7438
Chatbot found: English, Japanese | Token Rate: 16 | Complexity: 7.4068

Welcome to the BPA Chatbot Generator. Choose one of the options:
Press [1] to create random chatbots OR Press [2] to create a single chatbot
OR Press [3] to search by language OR Press [4] to end the program
3
Enter the language you want to search for:
z
No chatbot found for the specified language.
```

```
Welcome to the BPA Chatbot Generator. Choose one of the options:
Press [1] to create random chatbots OR Press [2] to create a single chatbot
OR Press [3] to search by language OR Press [4] to end the program
4
Goodbye!
```

User Entry Error Messages:

STEP 1 Entry Error

STEP 1: The user attempts to enter an invalid selection (number is not 1-4) the program will notify the user “Your entry is out of range”. If a letter or symbol value is entered the program will notify the user “ERROR: Please enter valid data type.” The program must return focus back to the main menu after the error message.

```
Welcome to the BPA Chatbot Generator. Choose one of the options:
Press [1] to create random chatbots OR Press [2] to create a single chatbot
OR Press [3] to search by language OR Press [4] to end the program
5
Your entry is out of range.

Welcome to the BPA Chatbot Generator. Choose one of the options:
Press [1] to create random chatbots OR Press [2] to create a single chatbot
OR Press [3] to search by language OR Press [4] to end the program
d
ERROR: Please enter valid data type.
```

```
Enter the token rate value (5 to 20):
w
ERROR: Please enter valid data type.
Enter the token rate value (5 to 20):
55
Your entry is out of range.
```

Option [2] Errors: There is no error handling for the language name entries. Error handling will occur when entering data for the Token Rate or the Complexity. The image to the right only shows the Token Rate. If the user attempts to enter a value out of range the program will notify the user “Your entry is out of range”. If a letter or

symbol value is entered the program will notify the user “ERROR: Please enter valid data type.” The program must return focus back to the menu after the error message. The program must return focus back to the exact location after the error message.

Output

You will program all the **printChatbots** method which oversees formatting and printing the randomly generated chatbots to the console. In this method, you will need to use the **PrintStream** class **printf()** formatting method when creating the table. Listed below are the format values for the record's width, alignment of each field header (the title at the top of the column) and placeholder data type:

- a. No. (this counts the number of bots): 4 for a string with left alignment
- b. Supported Languages: 30 for a string with left alignment
- c. Token Rate: 10 for a string with left alignment
- d. Complexity: 10 for a string with left alignment

The printing of the records will be similar, you will need to use the **PrintStream** class **printf()** formatting method. The values are similar but slightly different since there are numerical values being entered. Listed below are the format values for the record's width, alignment, and placeholder data type (remember these are the values for the data that is printed from the records):

- a. No. (this counts the number of bots): 4 for a integer with left alignment
- b. Supported Languages: 30 for a string with left alignment
- c. Token Rate: 10 for a integer with left alignment
- d. Complexity: 10 for a float with four decimal places and left alignment

Methods List

1. **main(String args[]):** This is the main method of the class where the program execution begins. It orchestrates the generation of chat bots, printing of chat bot details, and searching by language.
2. **printHighestComplexity():** This method prints the chat bot with the highest complexity rate along with its supported languages.
3. **printLowestComplexity():** This method prints the chat bot with the lowest complexity rate along with its supported languages.
4. **setChatBots(int bC):** This method generates and sets up chat bots with random attributes such as supported languages, token rate, and complexity.
5. **printChatBots():** This method prints all chat bots along with their supported languages, token rate, and complexity. It also calls **printHighestComplexity()** and **printLowestComplexity()**.
6. **inputManager():** This method manages user input to create chat bots, search by language, or exit the program.
7. **searchByLanguage():** This method allows the user to search for chat bots by specifying a language. It prints the details of chat bots that support the specified language.

Solution and Project (There is NO partial credit) (NOTE: UC represents uppercase and LC represents lowercase)		
The Java source file is present on the flash drive in a single folder with your contest ID		10 points
Program Execution (If the program does not execute, then the remaining items in this section receive a score of zero)		
Program loads and menu is present.		10 points
Program menu format matches the sample provided		10 points
STEP 1 Invalid Data Error is handled appropriately		20 points
STEP 1 Range Violation is handled appropriately		10 points
Option [1] is selected, program moves to appropriate STEP 2		10 points
STEP 2 Option [1] Invalid Data Error is handled appropriately		20 points
STEP 2 Option [1] Range Violation is handled appropriately		10 points
STEP 3 Option [1] correct number of random chatbots generated with all fields.		30 points
STEP 3 Option [1] program finds/prints the chatbot with “Most Expensive Complexity		30 points
STEP 3 Option [1] program finds/prints the chatbot with “Least Expensive Complexity		30 points
STEP 3 Option [1] format of output matches sample provided		20 points
After STEP 3 Option [1] Program returns immediately to active STEP 1		10 points
Option [2] is selected, program moves to appropriate STEP 2		10 points
STEP 2 Option [2] User can enter in Primary and Secondary Languages		10 points
STEP 3 Option [2] Token Rate Invalid Data Error is handled appropriately		20 points
STEP 3 Option [2] Token Rate Range Violation is handled appropriately		10 points
STEP 3 Option [2] Complexity Invalid Data Error is handled appropriately		20 points
STEP 3 Option [2] Complexity Range Violation is handled appropriately		10 points
STEP 4 Option [2] once data is properly entered program displays all bots and custom bot appears at the end of list.		30 points
After STEP 4 Option [2] Program returns immediately to active STEP 1		10 points
Option [3] is selected, program moves to appropriate STEP 2		10 points
STEP 2 Option [3] user can search for languages that have any sub string instance of their entry and returns all languages that meet the search.		30 points
After STEP 2 Option [3] Program returns immediately to active STEP 1		10 points
STEP 1 Option [4] program exits with goodbye message.		10 points
		20 points
Subtotal		/420 Points

Source Code Review No credit will be given for missing flagged comments. Code segments must work to receive full credit.		
A comment containing the contestant number is present at the top of the AIChatBotGenerator_State.java file		10 points
SC1: Flag Comment Present		5 points
SC1: Chatbots.java class: All constructors, methods, and variables are appropriately created		35 points
SC2: Flag Comment Present		5 points
SC2: AIChatBotGenerator_State class: create Scanner and ArrayList objects (place SC2 by both instances)		10 points
SC3: Flag Comment Present		5 points
SC3: AIChatBotGenerator_State class <i>printHighestComplexity()</i> : code that compares GPA's for all chatbots		15 points
SC4: Flag Comment Present		5 points
SC4: AIChatBotGenerator_State class <i>printLowestComplexity()</i> : code that compares GPA's for all chatbots		15 points
SC5: Flag Comment Present		5 points
SC5: AIChatBotGenerator_State class <i>inputManager()</i> : code with try/catch exception for invalid data error and range violation of STEP 1 (place SC5 by both instances)		15 points
SC6: Flag Comment Present		5 points
SC6: AIChatBotGenerator_State class <i>inputManager()</i> : code with try/catch exception for invalid data error and range violation of STEP 2 Option[1] (place SC6 by both instances)		15 points
SC7: Flag Comment Present		5 points
SC7: AIChatBotGenerator_State class <i>inputManager()</i> : code with try/catch exception for invalid INTEGER data error and range violation of STEP 3 Option[2] (place SC7 by both instances)		15 points
SC8: Flag Comment Present		5 points
SC8: AIChatBotGenerator_State class <i>inputManager()</i> : code with try/catch exception for invalid DOUBLE data error and range violation of STEP 3 Option[2] (place SC8 by both instances)		15 points
SC9: Flag Comment Present		5 points
SC9: AIChatBotGenerator_State class <i>inputManager()</i> : code that formats complexity with proper decimal format object (place SC9 by all uses of this object)		10 points
SC10: Flag Comment Present		5 points
SC10: AIChatBotGenerator_State class <i>inputManager()</i> : code that formats primary and secondary languages for Option [2] to required convention (first letter is UC, all other letters are LC)		15 points
SC11: Flag Comment Present		5 points
SC11: AIChatBotGenerator_State class <i>printChatBots()</i> : code uses <i>printf()</i> with all the required setting for spacing and values (see Output section for values). You must have the correct values for the field headers and the field values for the records.		35 points
Subtotal		/260 Points
Total Points		/680 Points